SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0309	**Course Title:**	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	**Total Marks:**	100
CO4 Statement:	*Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.*		
Student Name:	VAIBHAVA LAKSHMI.K	**Reg. No.:**	192511310
Section / Batch:	2025-2029	**Date:**	28/08/2026

Code of Conduct

I VAIBHAVA LAKSHMI.K (192511310) *(Name / Reg No)* *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Vaibhavi.K

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
	GRAND TOTAL:		100 Marks

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
   a) Clustering effect
   b) Memory usage
   c) Performance under high load factor
   d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
   a) Clustering issues
   b) Collision resolution efficiency
   c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
   a) Practical significance
   b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
   a) Space complexity (stack usage)
   b) Ease of implementation
   c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1) Comparison of Linear Probing and Separate chaining in hashing.

* Hashing is a technique used to store and retrieve data efficiently using hash table. When two keys produce the same hash index, a collision occurs.

Basis	Linear Probing	Separate chaining
a) Clustering effect	Suffers from primary clustering	Does not suffer from 1° clustering because collided elements are stored in separate
b) Memory usage	Uses memory to store inside hash table. No extra pointers are required.	Requires extra memory for linked lists or other structures used to store collided elements
c) Performance at high load factor.	Performance decreases significantly when the load factor becomes high.	Performs relatively better at high load factors because multiple elements can be stored in same chain.
d) Large-scale applications.	Suitable when load factor is kept low and memory locality is important.	Generally more suitable for large-scale app because it handles collisions..

→ a) Clustering effect:

⇒ Linear Probing:

If a collision occurs, the algorithm checks the next available position:

⇒ Separate chaining:

Each hash-table position maintains a chain, usually a linked list.

→ b) Memory Usage:

* Linear Probing is more memory-efficient because it stores all elements directly in the hash table. It does not require additional linked-list nodes & pointers.

* Separate Chaining requires extra memory because each bucket may contain linked list or another dynamic data structure.

Linear Probing - Low extra memory.
Separate Chaining - Higher memory usage.

→ c) Performance Under High load Factor:

* The load factor is:

$$\alpha = \frac{\text{Number of elements}}{\text{Size of hash table.}}$$

⇒ When the load factor is close to 1, linear probing can become inefficient because finding an empty position requires many probes.

→ d) Which is more suitable for Large-Scale Applications?

* Separate chaining is generally more suitable for large-scale applications.

Conclusion:

* Linear Probing is simple & memory-efficient, but it suffers from clustering & its performance decreases at high load factors.

* Separate chaining uses additional memory but provides better collision handling & more stable performance.

2) Quick Sort vs Merge sort for large - linked list.

* Suppose we need to sort linked list containing 10 million nodes.

Factor	Quick Sort	Merge Sort
Time complexity - Best	$O(n \log n)$	$O(n \log n)$
Time complexity - Average	$O(n \log n)$	$O(n \log n)$
Time complexity - Worst	$O(n^2)$	$O(n \log n)$
In-place	Can be implemented in-place	For linked lists, merging can be performed without copying.
Linked-list suitability	Less suitable	Highly suitable.

Why Merge sort is preferable for linked list.

* Linked lists do not support efficient random access. Quick Sort normally benefits from accessing elements around a pivot, whereas Merge sort only requires sequential traversal.

List 1: 10 → 30 → 50

List 2: 20 → 40 → 60

Merged: 10 → 20 → 30 → 40 → 50 → 60

Arrays:

* For arrays, the answer changes somewhat. Quick Sort is often preferred for arrays in practial application because:
  - It can operate in-place
  - It has good cache locality.
* However, merge sort is preferrable when:
  - Guaranteed O(n log n) worst-case is required.
  - Stability is important.

Conclusion:

* Linked list → Merge sort is preferable with sequential access.
* Array → Quick sort is often preferred in pratise.

Quadratic probing vs Double Hashing:

* Quadratic probing and double hashing are open-addressing collision-resolution techniques.

Basis	Quadratic Probing	Double Hashing
a) Clustering	Eliminates 1° clustering but can suffer from 2° clustering	Minimises both 1° & 2° clustering
b) Collision resolution efficiency	Better than linear probing	Generally more efficient
c) Dense hash tables	Performance decreases	Performs better in dense tables
Complexity	Average O(1), worst O(n)	Average O(1), worst O(n)

a) Clustering issues:

Instead of checking consecutive positions, it uses quadratic increments:

$h(k), h(k)+1^2, h(k)+2^2, h(k)+3^2, \ldots$

⇒ Double hashing:

It uses two hash functions:

$h(k,i) = h_1(k) + i \times h_2(k)) \bmod m$

b) Collision Resolution Efficiency:

* Double hashing is generally more efficient because the second hash function determines the step size.

Double hashing > Quadratic probing.

c) Suitable for Dense Hash Tables:

* As the hash table becomes dense, collisions become more frequent.

* Quadratic probing may experience 2° clustering & longer probe sequence.

Conclusion:

• Double Hashing is generally preferable to Quadratic probing, especially when the hash table has high load factor & minimising clustering.

Best Case vs Worst-Case time complexity of Sorting Algorithms:

Aspect	Best Case	Worst Case.
Meaning	Minimum time required for favourable input	Maximum time required for unfavourable I/P.
Practical significance	Shows how an algorithm perform	Shows the guarantee when I/P is difficult.
Quick Sort	$O(n \log n)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$
Insertion Sort	$O(n)$	$O(n^2)$

a) Practical significance:

* Best-case complexity tells us how an algorithm can perform when input is favourable.

$$Best = O(n)$$

* Worst-case complexity tells us the maximum amount of time an algorithm may require.

$$Worst = O(n^2)$$

b) Impact on Algorithm Selection:

* Algorithm selection depends on the nature of the I/P & the performance requirements.

For ex:

- Quick sort: Avg $O(n \log n)$, but worst case
- Merge Sort: $O(n \log n)$ even in worst case.
- Insertion Sort: Excellent for small or sorted data.
- Heap Sort: $O(n \log n)$ worst case & requires little extra memory.

* If guaranteed performance is important, merge sort or Heap sort may be selected instead of basic quick sort.

Conclusion:

* Best-case complexity is useful for understanding potential performance of an algorithm, but worst-case complexity is usually more important for reliable algorithm selection, particularly for large datasets & critical applications.

Recursive vs Iterative Quick Sort:

Quick Sort can be implemented either recursively or iteratively.

Factor	Recursive Quick Sort	Iterative Quick Sort
Space complexity.	O(n log n) average; O(n) worst case due to recursion stack	O(log n) average. with explicit stack.
Stack usage	Uses the system call stack automatically	Uses a programmer-managed stack.
Ease of implementation	Easier & more natural	More complicated
Control over memory	Less control	Greater control.

a) Space complexity:

* In recursive quick sort, every recursive call creates a stack frame.

→ Balanced partitions:

$$\text{space} = O(\log n)$$

→ Unbalanced partitions:

$$\text{space} = O(n).$$

b) Ease of implementation:

Recursive quick sort is easier to implement.

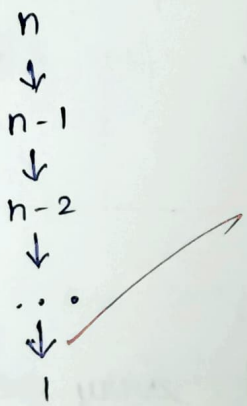
ex:

```
QuickSort ( left , right )
    partition
    QuickSort ( left part )
    QuickSort ( right part )
```

c) Risk of Stack Overflow:

Recursive Quick Sort can cause stack overflow when partitions become highly unbalanced.

for ex:



Conclusion:

Recursive Quick Sort is preferable for very large datasets or systems where stack overflow must be avoided, because it provides greater control over stack usage.